Пятое задание по программированию

Наивный байесовский классификатор

Базовая теория, разновидности алгоритма и практические сценарии применения

Содержательные акценты
• формула Байеса и предположение об условной независимости признаков
• варианты GaussianNB, MultinomialNB, BernoulliNB и ComplementNB
• примеры применения к учебным наборам данных и текстовой классификации

Далее приведен основной материал по теме с сохранением ключевых примеров, формул и программных фрагментов.

Формат представления: PDF-документ с теоретической частью и практическими иллюстрациями на Python.

Наивный байесовский классификатор

Наивный Байес: методы, примеры и практические задачи

1. Общая теория Наивного Байеса

Теорема Байеса: $P(Y|X) = \frac{P(Y)*P(X|Y)}{P(X)}$

Наивное предположение: признаки условно независимы в заданном классе

$P(X|Y) = \prod_{i=1}^{n} P(Xi|Y)$

Правило классификации: $\hat{y} = \arg\max_y P(y) \prod_{i=1}^{n} P(X_i|y)$

2. Методы Наивного Байеса

2.1. GaussianNB (Гауссов Наивный Байес)

Особенности:

- Для непрерывных признаков с нормальным распределением
- $P(x_i|y) \sim \mathcal{N}(\mu_y, \sigma_y^2)$

Пример: классификация ирисов

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Загрузка данных

X, y = load_iris(return_X_y=True)

# Разделение на обучающую и тестовую выборки

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.3, random_state=42

)

# Создание и обучение модели

gnb = GaussianNB()

gnb.fit(X_train, y_train)
```

```
y_pred = gnb.predict(X_test) # Предсказание
# Оценка модели
accuracy = accuracy_score(y_test, y_pred)
print(f"Точность: {accuracy:.2f}")
print(f"Количество ошибок: {(y_test != y_pred).sum()} из {len(y_test)}")
print("\nОтчет по классификации:")
print(classification_report(y_test, y_pred, target_names=load_iris().target_names))
# Просмотр параметров
print(f"Априорные вероятности классов: {gnb.class_prior_}")
print(f"Средние значения для каждого класса:\n{gnb.theta_}")
print(f"Дисперсии для каждого класса:\n{gnb.var_}")
```

```
Точность: 0.98 Количество ошибок: 1 из 45
Отчет по классификации:
              precision    recall  f1-score   support
      setosa       1.00      1.00      1.00        19
  versicolor       1.00      0.92      0.96        13
   virginica       0.93      1.00      0.96        13
    accuracy                           0.98        45
   macro avg       0.98      0.97      0.97        45
weighted avg       0.98      0.98      0.98        45
Априорные вероятности классов: [0.2952381  0.35238095 0.35238095]
Средние значения для каждого класса:
[[4.96451613 3.37741935 1.46451613 0.2483871 ]
 [5.86216216 2.72432432 4.21081081 1.3027027 ]
 [6.55945946 2.98648649 5.54594595 2.00540541]]
Дисперсии для каждого класса:
[[0.1119667  0.13658689 0.03325703 0.01152966]
 [0.27532506 0.08724617 0.23934259 0.04134405]
 [0.42241052 0.09630387 0.28842951 0.08591673]]
```

Задача 1: оцените качество модели на разных размерах тестовой выборки

```
def evaluate_gaussian(test_sizes):
    X, y = load_iris(return_X_y=True)
    results = {}
    for test_size in test_sizes:
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=test_size, random_state=42
        )
        gnb = GaussianNB()
        gnb.fit(X_train, y_train)
        y_pred = gnb.predict(X_test)
        accuracy = accuracy_score(y_test, y_pred)
        results[test_size] = accuracy
    return results
test_sizes = [0.2, 0.3, 0.4, 0.5]
results = evaluate_gaussian(test_sizes)
for size, acc in results.items():
    print(f"test_size={size}: точность={acc:.3f}")
```

test_size=0.2: точность=1.000

test_size=0.3: точность=0.978

test_size=0.4: точность=0.967

test_size=0.5: точность=0.987

2.2. MultinomialNB (Многочленный Наивный Байес)

Особенности:

- Для дискретных признаков (счетчики, частоты)
- Использует сглаживание Лапласа ($\alpha=1$) или Лидстоуна ($\alpha<1$)
- Идеален для классификации текстов

Пример: классификация текстов

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.pipeline import make_pipeline
import numpy as np
# Простой пример с текстами
texts = [
    'лучший фильм года отличный сюжет',
    'плохой фильм ужасная игра актеров',
    'отличная книга интересный сюжет',
    'ужасная книга скучно читать',
    'прекрасный фильм рекомендую всем',
    'не понравился фильм зря потратил время'
]
labels = [1, 0, 1, 0, 1, 0]  # 1 - положительный, 0 - отрицательный
# Создание пайплайна: векторизация + классификатор
model = make_pipeline(
    CountVectorizer(),
    MultinomialNB(alpha=1.0)  # сглаживание Лапласа
) # Обучение
model.fit(texts, labels)
# Предсказание для новых текстов
new_texts = [
    'отличный сюжет хорошая игра',
    'скучный и плохой фильм'
]
predictions = model.predict(new_texts)
probabilities = model.predict_proba(new_texts)
for text, pred, prob in zip(new_texts, predictions, probabilities):
    sentiment = "положительный" if pred == 1 else "отрицательный"
```

```
    print(f"Текст: '{text}'")
    print(f"Тональность: {sentiment}")
    print(f"Вероятности: положит={prob[1]:.3f}, отриц={prob[0]:.3f}\n")
# Просмотр важных признаков (слов)
feature_names = model.named_steps['countvectorizer'].get_feature_names_out()
class_probs = model.named_steps['multinomialnb'].feature_log_prob_
print("Наиболее важные слова для положительного класса:")
pos_words_idx = np.argsort(class_probs[1])[-10:]
for idx in pos_words_idx:
    print(f"  {feature_names[idx]}: {np.exp(class_probs[1][idx]):.3f}")
```

```
Текст: 'отличный сюжет хорошая игра'
Тональность: положительный
Вероятности: положит=0.780, отриц=0.220
Текст: 'скучный и плохой фильм'
Тональность: отрицательный
Вероятности: положит=0.358, отриц=0.642
Наиболее важные слова для положительного класса:
  отличный: 0.057
  интересный: 0.057
  всем: 0.057
  года: 0.057
  прекрасный: 0.057
  лучший: 0.057
  отличная: 0.057
  рекомендую: 0.057
  сюжет: 0.086
  фильм: 0.086
```

Задача 2: Сравните качество с разными значениями alpha

```
def compare_multinomial_alphas(alphas):
```

```
    texts = [
        'лучший фильм года отличный сюжет',
        'плохой фильм ужасная игра актеров',
        'отличная книга интересный сюжет',
        'ужасная книга скучно читать',
        'прекрасный фильм рекомендую всем',
        'не понравился фильм зря потратил время',
        'хороший сюжет отличная режиссура',
        'скучно неинтересно плохо'
    ]
    labels = [1, 0, 1, 0, 1, 0, 1, 0]
    # Разделение на обучение и тест
    X_train, X_test = texts[:6], texts[6:]
    y_train, y_test = labels[:6], labels[6:]
    results = {}
    for alpha in alphas:
        model = make_pipeline(
            CountVectorizer(),
            MultinomialNB(alpha=alpha)
        )
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        accuracy = accuracy_score(y_test, y_pred)
        results[alpha] = accuracy
    return results
alphas = [0.1, 0.5, 1.0, 2.0, 5.0]
results = compare_multinomial_alphas(alphas)
for alpha, acc in results.items():
    print(f"alpha={alpha}: точность={acc:.3f}")
```

alpha=0.1: точность=1.000

alpha=0.5: точность=1.000

alpha=1.0: точность=1.000

alpha=2.0: точность=1.000

alpha=5.0: точность=1.000

2.3. ComplementNB (Дополнение Наивного Байеса)

Особенности:

- Использует статистику из дополнения каждого класса
- Лучше работает с несбалансированными данными
- Более стабильные оценки параметров

Пример: работа с несбалансированными данными

```
from sklearn.naive_bayes import ComplementNB, MultinomialNB
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from collections import Counter
from sklearn.preprocessing import MinMaxScaler
import numpy as np
# Создание несбалансированного набора данных
X, y = make_classification(
    n_samples=1000,
    n_features=20,
    n_informative=15,
    n_redundant=5,
    weights=[0.9, 0.1],
    random_state=42
)
# Преобразование признаков для MultinomialNB (убираем отрицательные значения)
```

```
# Способ 1: Сдвиг всех значений на минимальное (если есть отрицательные)
X = X - X.min() + 0.1  # добавляем небольшую константу чтобы избежать нулей
print(f"Распределение классов: {Counter(y)}")
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
# Сравнение MultinomialNB и ComplementNB
mnb = MultinomialNB()
cnb = ComplementNB()
scaler = MinMaxScaler()
X = scaler.fit_transform(X)
mnb.fit(X_train, y_train)
cnb.fit(X_train, y_train)
mnb_pred = mnb.predict(X_test)
cnb_pred = cnb.predict(X_test)
print(f"MultinomialNB точность: {accuracy_score(y_test, mnb_pred):.3f}")
print(f"ComplementNB точность: {accuracy_score(y_test, cnb_pred):.3f}")
# Детальный анализ для минорного класса
mnb_metrics = precision_recall_fscore_support(y_test, mnb_pred, average=None)
cnb_metrics = precision_recall_fscore_support(y_test, cnb_pred, average=None)
print("\nМетрики для минорного класса (класс 1):")
print(f"MultinomialNB - precision={mnb_metrics[0][1]:.3f}, recall={mnb_metrics[1][1]:.3f}")
print(f"ComplementNB  - precision={cnb_metrics[0][1]:.3f}, recall={cnb_metrics[1][1]:.3f}")
mnb_pred = mnb.predict(X_test)
cnb_pred = cnb.predict(X_test)
print(f"MultinomialNB точность: {accuracy_score(y_test, mnb_pred):.3f}")
print(f"ComplementNB точность: {accuracy_score(y_test, cnb_pred):.3f}")
```

```
# Детальный анализ для минорного класса
from sklearn.metrics import precision_recall_fscore_support
mnb_metrics = precision_recall_fscore_support(y_test, mnb_pred, average=None)
cnb_metrics = precision_recall_fscore_support(y_test, cnb_pred, average=None)
print("\nМетрики для минорного класса (класс 1):")
print(f"MultinomialNB - precision={mnb_metrics[0][1]:.3f},
recall={mnb_metrics[1][1]:.3f}")
print(f"ComplementNB  - precision={cnb_metrics[0][1]:.3f},
recall={cnb_metrics[1][1]:.3f}")
```

```
Распределение классов: Counter({np.int64(0): 898, np.int64(1): 102})
MultinomialNB точность: 0.867
ComplementNB точность: 0.640

Метрики для минорного класса (класс 1):
MultinomialNB - precision=0.200, recall=0.097
ComplementNB  - precision=0.171, recall=0.645
MultinomialNB точность: 0.867
ComplementNB точность: 0.640

Метрики для минорного класса (класс 1):
MultinomialNB - precision=0.200, recall=0.097
ComplementNB  - precision=0.171, recall=0.645
```

2.4. BernoulliNB (Бернулли Наивный Байес)

Особенности: для бинарных признаков (0/1). Подходит для классификации коротких текстов. Использует бинаризацию признаков

Пример: классификация с бинарными признаками

```
from sklearn.naive_bayes import BernoulliNB
from sklearn.feature_extraction.text import CountVectorizer
# Данные
texts = [
```

```
    'спам предложение купить товар',
    'важное письмо от начальника',
    'купить дешево виагра',
    'отчет за месяц',
    'заработок в интернете',
    'совещание в пятницу'
]
labels = [1, 0, 1, 0, 1, 0]  # 1 - спам, 0 - не спам
# Пайплайн с BernoulliNB
model = make_pipeline(
    CountVectorizer(binary=True),  # Важно: binary=True для бинарных признаков
    BernoulliNB(alpha=1.0, binarize=0.0)  # binarize порог
)
model.fit(texts, labels)
# Тестирование
test_texts = [
    'купить дешево',
    'встреча с начальником'
]
predictions = model.predict(test_texts)
probabilities = model.predict_proba(test_texts)
for text, pred, prob in zip(test_texts, predictions, probabilities):
    print(f"Текст: '{text}'")
    print(f"Класс: {'спам' if pred == 1 else 'не спам'}")
    print(f"Вероятность спама: {prob[1]:.3f}\n")
```

Текст: 'купить дешево'. Класс: спам. Вероятность спама: 0.934

Текст: 'встреча с начальником'. Класс: не спам. Вероятность спама: 0.471

Задача 3: Сравните BernoulliNB с MultinomialNB для коротких текстов

```
# Сравнение BernoulliNB и MultinomialNB для коротких текстов
short_texts = [
    'купить', 'дешево', 'виагра', 'заработок', 'деньги',  # спам-слова
    'встреча', 'отчет', 'письмо', 'начальник', 'план'      # не спам
]
labels = [1, 1, 1, 1, 1, 0, 0, 0, 0, 0]
# Создание тестовых фраз
train_texts = [
    'купить дешево', 'заработок деньги', 'виагра купить',  # спам
    'встреча план', 'отчет письмо', 'начальник встреча'    # не спам
]
train_labels = [1, 1, 1, 0, 0, 0]
test_texts = ['дешево виагра', 'отчет начальник']
test_labels = [1, 0]
# MultinomialNB
mnb_model = make_pipeline(CountVectorizer(), MultinomialNB())
mnb_model.fit(train_texts, train_labels)
# BernoulliNB
bnb_model = make_pipeline(CountVectorizer(binary=True), BernoulliNB())
bnb_model.fit(train_texts, train_labels)
print("MultinomialNB:")
print(f"  Точность: {accuracy_score(test_labels,
mnb_model.predict(test_texts)):.3f}")
print("BernoulliNB:")
print(f"  Точность: {accuracy_score(test_labels,
bnb_model.predict(test_texts)):.3f}")
```

```
MultinomialNB: Точность: 1.000
BernoulliNB: Точность: 1.000
```

2.5. CategoricalNB (Категориальный Наивный Байес)

Особенности:

- Для категориальных признаков
- Требует числового кодирования категорий
- Оценивает вероятности для каждой категории

Пример: классификация по категориальным признакам

```
from sklearn.naive_bayes import CategoricalNB
from sklearn.preprocessing import OrdinalEncoder
import pandas as pd
# Создание категориальных данных
# Признаки: [Образование, Доход, Город]
data = [
    ['высшее', 'высокий', 'москва'],
    ['среднее', 'средний', 'спб'],
    ['высшее', 'средний', 'москва'],
    ['среднее', 'низкий', 'казань'],
    ['высшее', 'высокий', 'спб'],
    ['среднее', 'средний', 'москва']
]
target = [1, 0, 1, 0, 1, 0]  # 1 - купил товар, 0 - не купил
# Кодирование категориальных признаков
encoder = OrdinalEncoder()
X_encoded = encoder.fit_transform(data)
# Обучение модели
cnb = CategoricalNB(alpha=1.0)
cnb.fit(X_encoded, target)
# Предсказание для новых данных
new_data = [
    ['высшее', 'средний', 'казань'],
    ['среднее', 'низкий', 'спб']
```

```
]
new_data_encoded = encoder.transform(new_data)
predictions = cnb.predict(new_data_encoded)
probabilities = cnb.predict_proba(new_data_encoded)
# Вывод результатов с расшифровкой
for i, (sample, pred, prob) in enumerate(zip(new_data, predictions, probabilities)):
    print(f"Клиент {i+1}: {sample}")
    print(f"  Предсказание: {'купит' if pred == 1 else 'не купит'}")
    print(f"  Вероятность покупки: {prob[1]:.3f}")
# Просмотр вероятностей для каждой категории
print("\nВероятности категорий для признака 'Образование':")
feature_idx = 0
for class_idx in range(2):
    print(f"Класс {class_idx}:")
    probs = np.exp(cnb.feature_log_prob_[class_idx][feature_idx])
    categories = encoder.categories_[feature_idx]
    for cat, prob in zip(categories, probs):
        print(f"  {cat}: {prob:.3f}")
```

```
Клиент 1: ['высшее', 'средний', 'казань']
  Предсказание: купит
  Вероятность покупки: 0.571
Клиент 2: ['среднее', 'низкий', 'спб']
  Предсказание: не купит
  Вероятность покупки: 0.111
Вероятности категорий для признака 'Образование':
Класс 0: высшее: 0.200 среднее: 0.800
Класс 1: высшее: 0.167 среднее: 0.333
```

Задача 4: Анализ влияния сглаживания alpha на редкие категории

```
# Данные с редкими категориями
```

```
data = [
    ['A', 'X'], ['A', 'X'], ['A', 'X'], ['A', 'Y'],
    ['B', 'X'], ['B', 'Y'], ['B', 'Y'], ['B', 'Z'],  # Z - редкая категория
    ['C', 'X'], ['C', 'Y']
]
target = [0, 0, 0, 1, 1, 1, 1, 0, 0, 1]
encoder = OrdinalEncoder()
X_encoded = encoder.fit_transform(data)
# Тестовый образец с редкой категорией
test_sample = [['B', 'Z']]
test_encoded = encoder.transform(test_sample)
alphas = [0.01, 0.1, 1.0, 10.0]
for alpha in alphas:
    cnb = CategoricalNB(alpha=alpha)
    cnb.fit(X_encoded, target)
    prob = cnb.predict_proba(test_encoded)[0]
    print(f"alpha={alpha:.2f}: P(класс=0)={prob[0]:.3f},
P(класс=1)={prob[1]:.3f}")
```

```
alpha=0.01: P(класс=0)=0.971, P(класс=1)=0.029
alpha=0.10: P(класс=0)=0.796, P(класс=1)=0.204
alpha=1.00: P(класс=0)=0.500, P(класс=1)=0.500
alpha=10.00: P(класс=0)=0.482, P(класс=1)=0.518
```

2.6. Out-of-core обучение (partial_fit)

Особенности:

- Для работы с данными, не помещающимися в память
- Постепенное обучение на батчах
- Требует указания всех классов при первом вызове

Пример: пакетное обучение на больших данных

```
from sklearn.naive_bayes import MultinomialNB
```

```
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
# Имитация потока данных
def generate_data_batches(n_batches=5, batch_size=10):
    classes = [0, 1]
    vectorizer = CountVectorizer()
    for batch_num in range(n_batches):
        # Генерация текстов для батча
        texts = []
        labels = []
        for i in range(batch_size):
            if np.random.random() > 0.5:
                texts.append(f"спам предложение купить товар {batch_num}_{i}")
                labels.append(1)
            else:
                texts.append(f"обычное письмо отчет {batch_num}_{i}")
                labels.append(0)
        # Векторизация для этого батча
        if batch_num == 0:
            X_batch = vectorizer.fit_transform(texts)
        else:
            X_batch = vectorizer.transform(texts)
        yield X_batch, np.array(labels)
    return vectorizer
# Инкрементальное обучение
model = MultinomialNB()
classes = np.array([0, 1])
# Первый батч - нужно указать все классы
first_batch = True
```

```
batch_sizes = []
for X_batch, y_batch in generate_data_batches(n_batches=5, batch_size=20):
    if first_batch:
        model.partial_fit(X_batch, y_batch, classes=classes)
        first_batch = False
    else:
        model.partial_fit(X_batch, y_batch)
    batch_sizes.append(X_batch.shape[0])
    print(f"Обработан батч размером {X_batch.shape[0]}")
print(f"\nВсего обработано {sum(batch_sizes)} образцов")
# Тестирование финальной модели
test_texts = ['купить дешево', 'отчет за месяц']
test_vectorizer = generate_data_batches.__defaults__[0]  # Получаем vectorizer
# Примечание: в реальном коде нужно сохранить vectorizer
```

Обработан батч размером 20

Обработан батч размером 20

Обработан батч размером 20

Обработан батч размером 20

Обработан батч размером 20

Всего обработано 100 образцов

Задача 5: Сравните качество при разных размерах батча

```
def compare_batch_sizes(batch_sizes):
    results = {}
    for batch_size in batch_sizes:
        model = MultinomialNB()
        classes = np.array([0, 1])
        first = True
        for X_batch, y_batch in generate_data_batches(n_batches=10,
batch_size=batch_size):
```

```
            if first:
                model.partial_fit(X_batch.toarray(), y_batch, classes=classes)
                first = False
            else:
                model.partial_fit(X_batch.toarray(), y_batch)
        # Тестирование на отложенной выборке
        X_test, y_test = next(generate_data_batches(n_batches=1, batch_size=50))
        y_pred = model.predict(X_test.toarray())
        accuracy = accuracy_score(y_test, y_pred)
        results[batch_size] = accuracy
    return results
# Примечание: для запуска потребуется адаптировать функцию generate_data_batches
# для работы с питру массивами
```

3. Сравнительный анализ методов

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, load_iris
from sklearn.model_selection import cross_val_score
# Сравнение всех методов на разных типах данных
datasets = {
    'iris': load_iris(return_X_y=True),
    'binary': make_classification(n_samples=500, n_features=10, random_state=42),
    'imbalanced': make_classification(n_samples=500, weights=[0.9, 0.1], random_state=42)
}
classifiers = {
    'GaussianNB': GaussianNB(),
    'MultinomialNB': MultinomialNB(),
    'ComplementNB': ComplementNB(),
```

```
    'BernoulliNB': BernoulliNB(binarize=0.0),
    'CategoricalNB': CategoricalNB()
}
results = {}
for dataset_name, (X, y) in datasets.items():
    results[dataset_name] = {}
    # Для Multinomial, Complement, Bernoulli нужно преобразование в неотрицательные значения
    if dataset_name != 'iris':
        X = X - X.min() + 1  # Сдвиг для положительных значений
    for clf_name, clf in classifiers.items():
        # CategoricalNB требует категориальные данные
        if clf_name == 'CategoricalNB' and dataset_name != 'categorical':
            continue
        scores = cross_val_score(clf, X, y, cv=5, scoring='accuracy')
        results[dataset_name][clf_name] = {
            'mean': scores.mean(),
            'std': scores.std()
        }
# Вывод результатов
for dataset_name, clf_results in results.items():
    print(f"\nДатасет: {dataset_name}")
    for clf_name, metrics in clf_results.items():
        print(f"  {clf_name}: {metrics['mean']:.3f} ± {metrics['std']:.3f}")
```

Датасет: iris GaussianNB: 0.953 ± 0.027 MultinomialNB: 0.953 ± 0.045 ComplementNB: 0.667 ± 0.000 BernoulliNB: 0.333 ± 0.000

Датасет: binary GaussianNB: 0.890 ± 0.030 MultinomialNB: 0.856 ± 0.010 ComplementNB: 0.856 ± 0.010 BernoulliNB: 0.500 ± 0.000

Датасет: imbalanced GaussianNB: 0.940 ± 0.030 MultinomialNB: 0.896 ± 0.005

ComplementNB: 0.830 ± 0.023 BernoulliNB: 0.896 ± 0.005

4. Практические рекомендации

A	B	C	D
Метод	*Тип данных*	*Когда использовать*	*Особенности*
GaussianNB	Непрерывные	Признаки распределены нормально	Быстро, хорошо для числовых данных
MultinomialNB	Дискретные (счетчики)	Классификация текстов, tf-idf	Требует неотрицательные признаки
ComplementNB	Дискретные	Несбалансированные классы	Лучше MNB для редких классов
BernoulliNB	Бинарные	Короткие тексты, признаки присутствия	Явно учитывает отсутствие признаков
CategoricalNB	Категориальные	Категориальные признаки	Требует числового кодирования

Выводы:

1. Наивный Байес работает быстро и эффективно на больших данных
2. Выбор метода зависит от типа признаков и задачи
3. Для несбалансированных данных предпочтительнее ComplementNB
4. Сглаживание (alpha) важно для работы с редкими категориями
5. Out-of-core обучение позволяет обрабатывать данные любого размера